

Highly Configurable Mechanical Accelerometer

Team 703: Cyclone Rocketry at Iowa State University

Design Context

Our payload is a highly configurable mechanical accelerometer. It consists of a mass hanging by 4 springs, whose location is recorded by a potentiometer to be stored as a voltage on an SD card. Then, post-processing is done to convert the voltage to the acceleration we are experiencing, using the following equation:

$$\text{Acceleration} = \frac{2 \times k \times \frac{V}{V_{dd}} \times L}{m}$$

Table 1: Variable Allocation

Name	Variable (Flight Configuration)
k	Spring Constant (0.339 Newton-cm)
V	Present Voltage
V_{dd}	Supply Voltage (3.3V)
L	Length of Potentiometer (10 cm)
m	Mass of cube (0.0523 kg)

Data Analysis

The following figure is a graphic of our data, as given by a post-processing python script provided in the appendix. This script concatenates each collected data file, which are a maximum size of 1 hour each, and plots the measured acceleration of the concatenated files. The result of this plotting is below. As a general heuristic, the lower that the mass is in the payload, the lower the voltage is, and therefore the greater the acceleration is along the recorded axis.

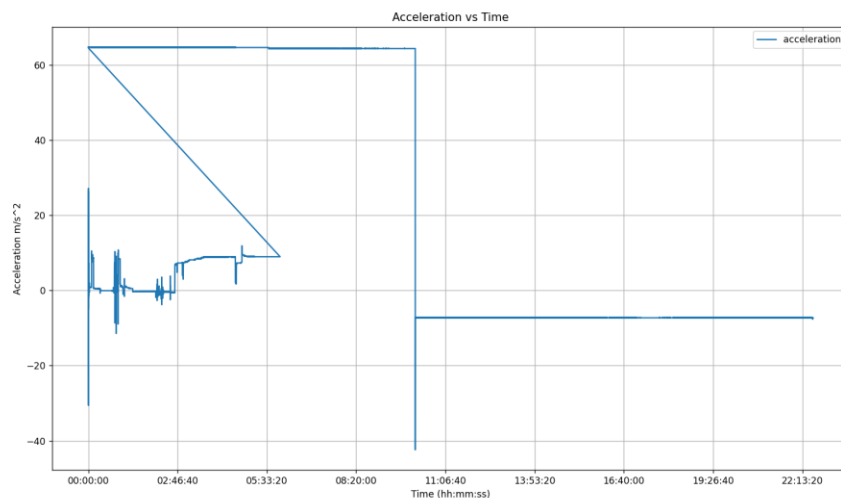


Figure 1: Full Flight Data

Here we can see that the data is simply incorrect, and further analysis of the data and timing has revealed to us what we think happened during the flight of *Hyperion*. Firstly, below we have a subset of the data points that we believe are relevant on the recorded flight.

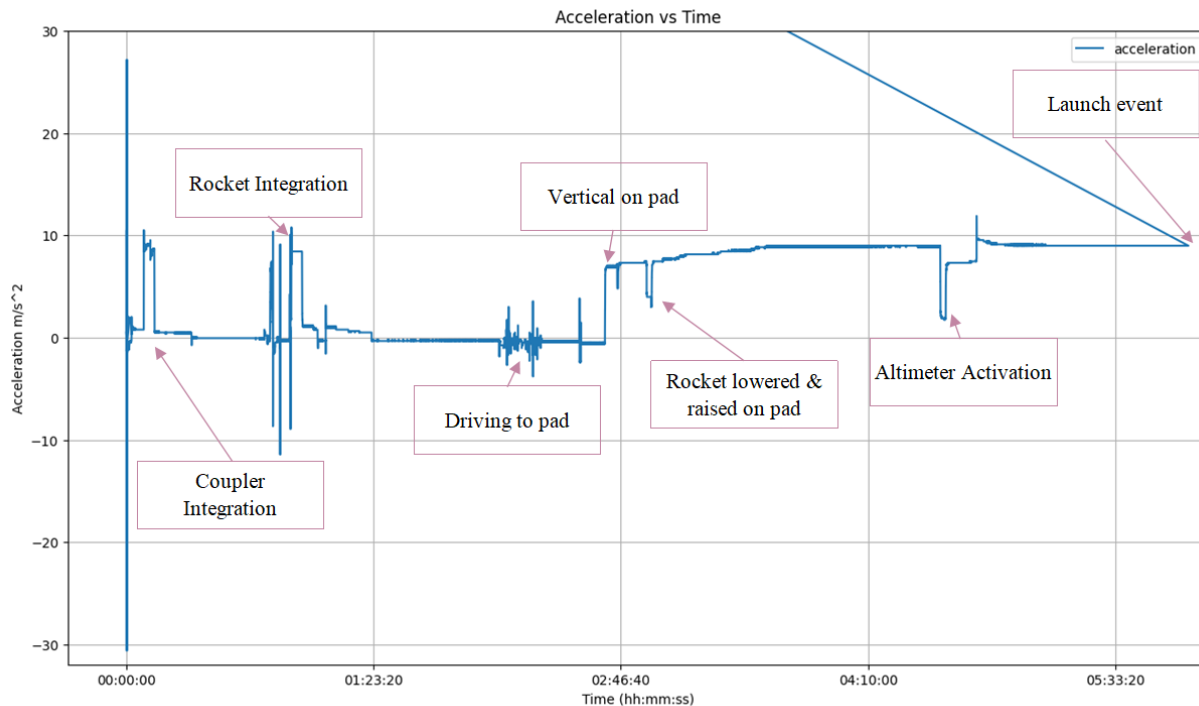


Figure 2: Labelled Preflight Plot

First, we powered on the payload and manually moved the mass up and down a bit and placed it into the coupler horizontally. Then, we integrated the rest of the rocket and did some “beep testing” of recovery electronics to ensure a nominal flight, with sometimes placing the rocket vertically to help “ease” the tubes around the couplers. This is why we can see the acceleration bouncing between 0 and 9.8 m/s^2 . Then, the accelerometer sat horizontally waiting to be taken out to the pad. We can see the drive as a series of bumps and bounces on the graph. Then, the rocket was placed vertically on the rail, and the acceleration became $\sim 9.8 \text{ m/s}^2$, as expected. An integration mistake caused the rocket to be lowered and raised again, and then we waited for launch. During this time, we can see that the mass seems to “settle” into place over some time. Then, the GPS altimeters were activated, so the rocket was lowered and raised again. Then, when we expect the launch event, we are no longer recording valid data.

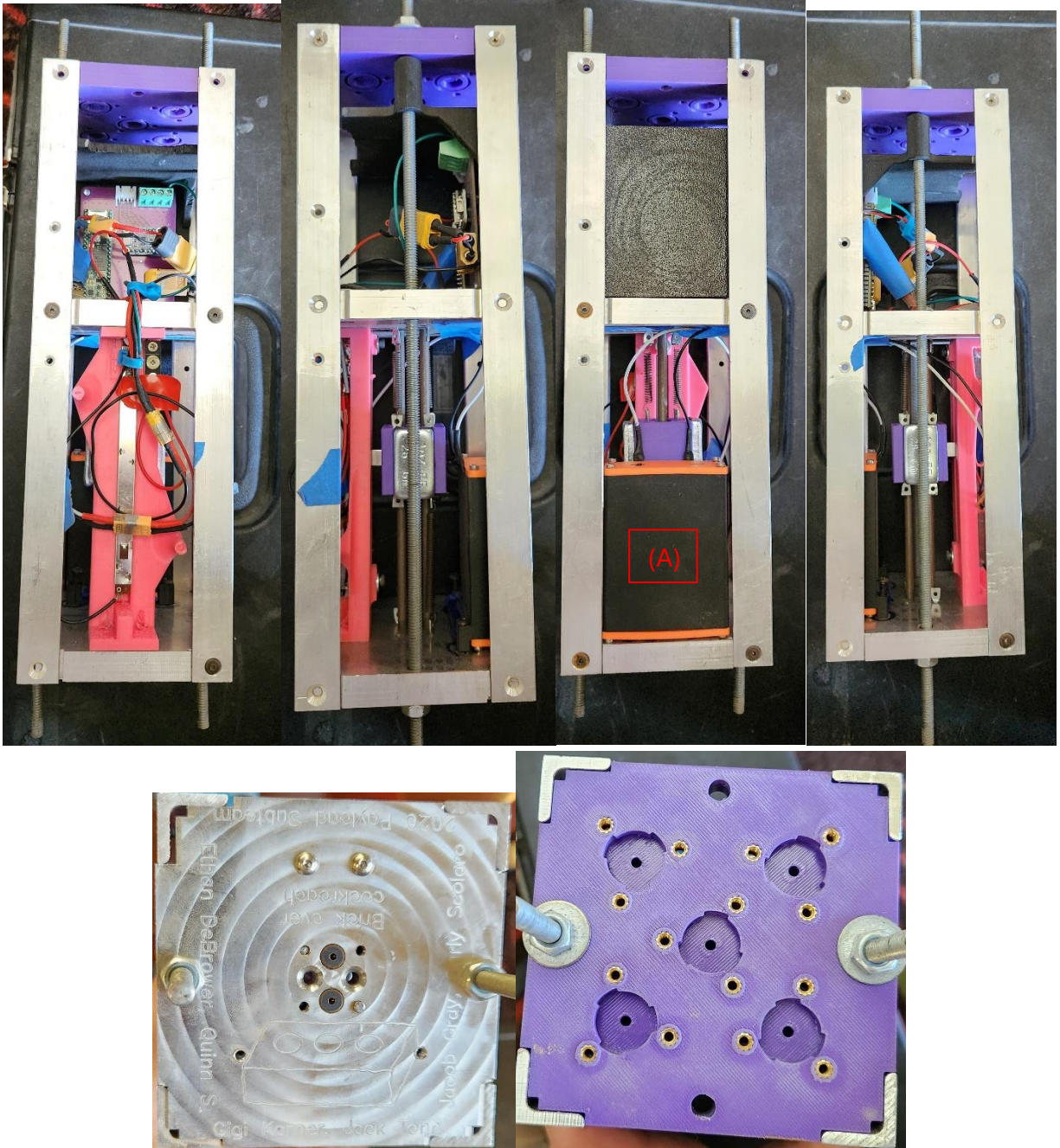
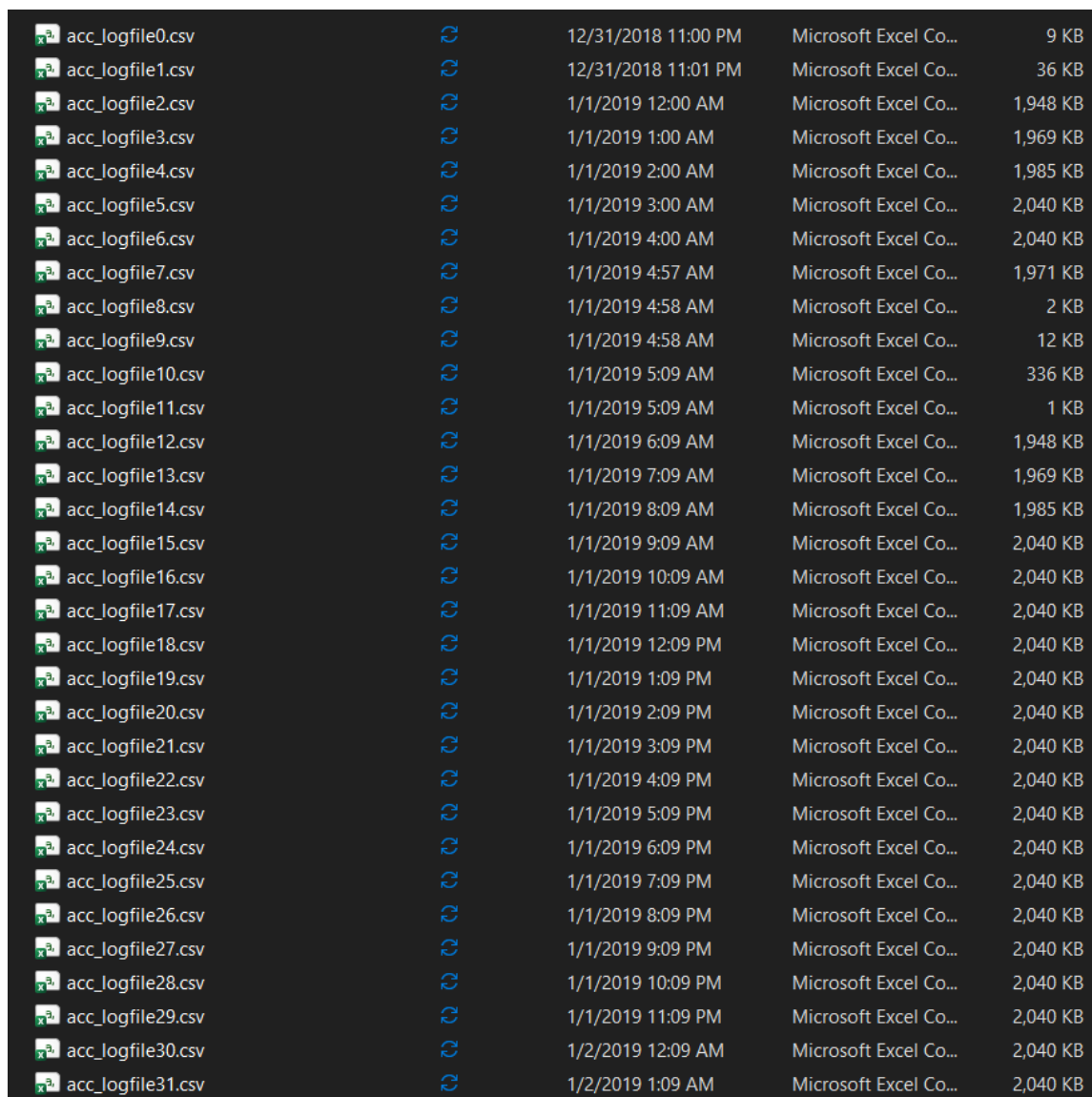


Figure 3: Post-Flight Payload

Above are pictures of the payload after flight, with the top 4 pictures being 90-degree rotations clockwise, and the bottom two being the bottom and top of the payload, respectively. Thus, when we launched the purple bulkhead was on top, so all acceleration would be from purple to metal bulkhead. We would like to take note of the black box labelled (A) in the 3rd picture. This is our battery box, which contains two 18650 cells in series. On the right side, there is a battery with the positive side down, and negative side up. Thus, with our battery connectors, there is a spring below the battery, and a static

mount on top, which means that during a launch event, it is possible (and likely) that the battery would slide down the box and break continuity. Because we neglected to put any capacitors on the VDD lines, when the battery breaks continuity, the entire system loses power.

Below is a screenshot of the datafiles. Pay special attention to the size column on the right side. The first two files are from testing battery operations on the day of the flight, and that is why they are smaller sizes. Then the next 6 files are the ones that are annotated above, with each file containing an hour of information.



	acc_logfile0.csv		12/31/2018 11:00 PM	Microsoft Excel Co...	9 KB
	acc_logfile1.csv		12/31/2018 11:01 PM	Microsoft Excel Co...	36 KB
	acc_logfile2.csv		1/1/2019 12:00 AM	Microsoft Excel Co...	1,948 KB
	acc_logfile3.csv		1/1/2019 1:00 AM	Microsoft Excel Co...	1,969 KB
	acc_logfile4.csv		1/1/2019 2:00 AM	Microsoft Excel Co...	1,985 KB
	acc_logfile5.csv		1/1/2019 3:00 AM	Microsoft Excel Co...	2,040 KB
	acc_logfile6.csv		1/1/2019 4:00 AM	Microsoft Excel Co...	2,040 KB
	acc_logfile7.csv		1/1/2019 4:57 AM	Microsoft Excel Co...	1,971 KB
	acc_logfile8.csv		1/1/2019 4:58 AM	Microsoft Excel Co...	2 KB
	acc_logfile9.csv		1/1/2019 4:58 AM	Microsoft Excel Co...	12 KB
	acc_logfile10.csv		1/1/2019 5:09 AM	Microsoft Excel Co...	336 KB
	acc_logfile11.csv		1/1/2019 5:09 AM	Microsoft Excel Co...	1 KB
	acc_logfile12.csv		1/1/2019 6:09 AM	Microsoft Excel Co...	1,948 KB
	acc_logfile13.csv		1/1/2019 7:09 AM	Microsoft Excel Co...	1,969 KB
	acc_logfile14.csv		1/1/2019 8:09 AM	Microsoft Excel Co...	1,985 KB
	acc_logfile15.csv		1/1/2019 9:09 AM	Microsoft Excel Co...	2,040 KB
	acc_logfile16.csv		1/1/2019 10:09 AM	Microsoft Excel Co...	2,040 KB
	acc_logfile17.csv		1/1/2019 11:09 AM	Microsoft Excel Co...	2,040 KB
	acc_logfile18.csv		1/1/2019 12:09 PM	Microsoft Excel Co...	2,040 KB
	acc_logfile19.csv		1/1/2019 1:09 PM	Microsoft Excel Co...	2,040 KB
	acc_logfile20.csv		1/1/2019 2:09 PM	Microsoft Excel Co...	2,040 KB
	acc_logfile21.csv		1/1/2019 3:09 PM	Microsoft Excel Co...	2,040 KB
	acc_logfile22.csv		1/1/2019 4:09 PM	Microsoft Excel Co...	2,040 KB
	acc_logfile23.csv		1/1/2019 5:09 PM	Microsoft Excel Co...	2,040 KB
	acc_logfile24.csv		1/1/2019 6:09 PM	Microsoft Excel Co...	2,040 KB
	acc_logfile25.csv		1/1/2019 7:09 PM	Microsoft Excel Co...	2,040 KB
	acc_logfile26.csv		1/1/2019 8:09 PM	Microsoft Excel Co...	2,040 KB
	acc_logfile27.csv		1/1/2019 9:09 PM	Microsoft Excel Co...	2,040 KB
	acc_logfile28.csv		1/1/2019 10:09 PM	Microsoft Excel Co...	2,040 KB
	acc_logfile29.csv		1/1/2019 11:09 PM	Microsoft Excel Co...	2,040 KB
	acc_logfile30.csv		1/2/2019 12:09 AM	Microsoft Excel Co...	2,040 KB
	acc_logfile31.csv		1/2/2019 1:09 AM	Microsoft Excel Co...	2,040 KB

Figure 4: Recorded Logfiles

CSVs 8, 9, 10, and 11 are all noticeably less than the expected 2000 KB for a full recording, and upon inspection we can see that they start at 0ms. To us, this suggests that at launch time, we did lose power, causing the system to reset. Then the payload sat at 0V for 10 hours, which is unexpected and is still being analyzed. Upon early inspection, it seems that the wiring may have shorted on our potentiometer analog output with ground, causing the 0V. Then, the payload was struck by rain and settled on a consistent voltage of around 1.5V.

Despite the lack of data from our launch, we still believe that we have a solid payload design, and if the battery box was rotated 90 degrees such that gravity would not shift the batteries, that we would collect interesting and useful data. One of the factors that give us faith in our design is that on June 17th, we attempted launch and we were in queue for around 11 hours. We recorded data that entire time, the plot of it is seen below.

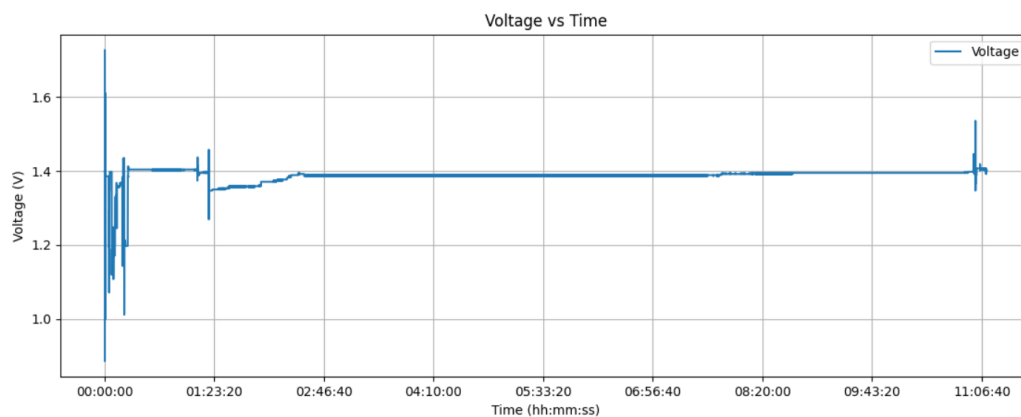


Figure 5: 6-17 Voltage

This furthers our confidence that the launch event caused our intermittent power, as the payload was able to run for 11 hours without our power cycling, and we started with fresh fully charged batteries for each day.

Conclusions and Lessons Learned

In conclusion, we have designed and manufactured an accelerometer that has a critical failure point at high accelerations. However, we still have full confidence in our design, and we believe that rotating the battery box will fix the problem of intermittent power on launch. We plan to fly this payload again later in the summer to prove to ourselves that it is functional as an accelerometer.

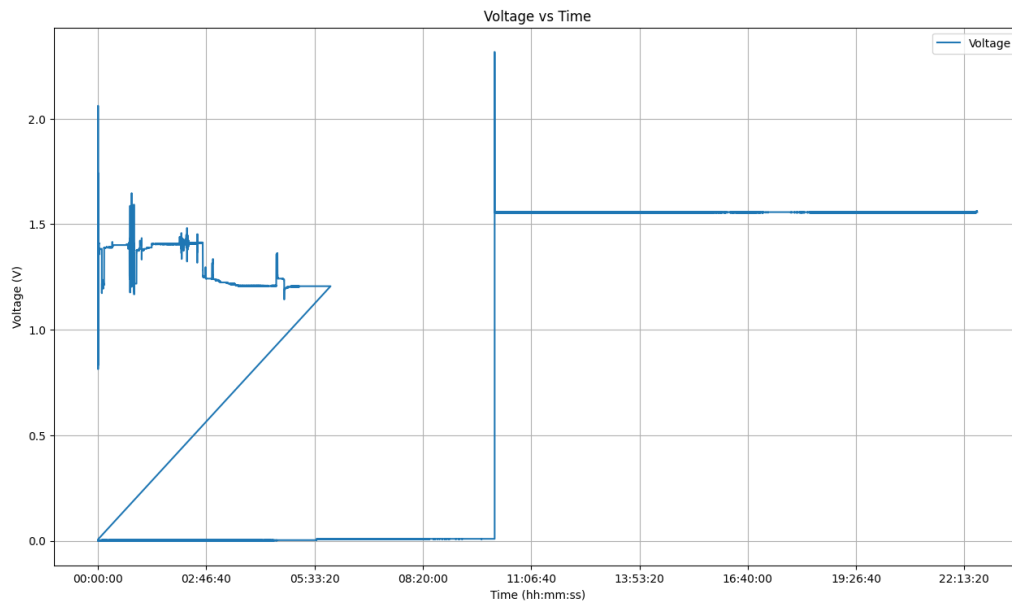
In the future, we recommend that teams have a checklist for each component designed, consisting of the following basic questions:

- What purpose does this component serve?
- Is this component manufacturable or purchasable?
 - What materials will be used?
- Where is this component going to be in the larger design?
 - How is it going to be attached?
 - How will the added forces of acceleration challenge assumptions for this component?

If these questions are seriously considered during the design process, they will help us to design a stronger payload design for future competitions.

Appendices

Voltage plot from flight:



Data collection C++ code:

```
#include <Arduino.h>
#include <Adafruit_ADS1X15.h>
#include <SD.h>

#define LOOP_TIMESTEP_MS 50
//writes per hour
#define MAX_NUM_WRITES ((1000/LOOP_TIMESTEP_MS) * 3600)

Adafruit_ADS1015 adc;
#define ADC_VCC (5)
#define ADC_POT_LENGTH_CM (10)
#define ADC_ADDR (0x48)
#define OUTPUT_CSV_FILENAME "acc_logfile"

const int chipSelect = BUILTIN_SDCARD;

String logfile_name;
File logfile;
char print_buffer[50];
```

```

int filename_number = 0;
int num_writes = 0;

// put function declarations here:
float volts_to_position(float volts);
void create_file();

void setup() {
    Serial.begin(9600);
    Serial.println("Waddup");

    //adc init
    if(!adc.begin(ADC_ADDR, &Wire2)) {
        Serial.println("ADC not ackd ");
        while(1);
    }

    //spi/sd init
    if (!SD.begin(chipSelect)) {
        Serial.println("Card failed, or not present");
        while(1);
    }
    Serial.println("card initialized.");

    //logfile init
    filename_number = 0;
    create_file();
}

void loop() {

    int curr_time = millis();

    float volts0 = adc.computeVolts(adc.readADC_SingleEnded(0)); //read channel
    float pos_x_cm = volts_to_position(volts0);
    sprintf(print_buffer, "%d, %f, %f\n", curr_time, volts0, pos_x_cm);
    Serial.printf("%s", print_buffer);

    if (logfile) {
        logfile.print(print_buffer);
        logfile.flush();
        ++num_writes;
    } else {
        Serial.println("error logfile.txt");
    }
}

```

```

}

if(num_writes >= MAX_NUM_WRITES) {
    ++filename_number;
    num_writes = 0;
    create_file();
}

//wait for timestep so each data point is similar
while ( (millis() - curr_time) < LOOP_TIMESTEP_MS);
}

float volts_to_position(float volts){
    float position = (volts/ADC_VCC)*ADC_POT_LENGTH_CM;
    return position;
}

void create_file(){
    logfile_name = OUTPUT_CSV_FILENAME + String(filename_number) + ".csv";
    while(SD.exists(logfile_name.c_str())){
        ++filename_number;
        logfile_name = OUTPUT_CSV_FILENAME + String(filename_number) + ".csv";
    }
    logfile = SD.open(logfile_name.c_str(), FILE_WRITE);
    if(logfile) {
        logfile.print("Time,Voltage,Pos\n");
    } else {
        Serial.print("Logfile error");
        while(1);
    }
}
}

```

Post processing Python script:

```

import matplotlib
matplotlib.use("TkAgg")

import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
import sys

old_voltage = 5.0
new_voltage = 2.8

```

```

pot_length = 10.0
mass = 0.0523
spring_const = 0.339

def format_time(x, pos):
    total_seconds = x / 1000.0
    hours = int(total_seconds // 3600)
    minutes = int((total_seconds % 3600) // 60)
    seconds = int(total_seconds % 60)
    return f"{hours:02}:{minutes:02}:{seconds:02}"

if len(sys.argv) < 2:
    print("Usage: python plot.py <csv_file>")
    sys.exit(1)

filename = sys.argv[1]

df = pd.read_csv(filename)

time = df["Time"]

# Plot voltage
plt.figure()
for col in df.columns:
    if "Voltage" in col:
        plt.plot(time, df[col], label=col)

plt.gca().xaxis.set_major_formatter(FuncFormatter(format_time))

plt.title("Voltage vs Time")
plt.xlabel("Time (hh:mm:ss)")
plt.ylabel("Voltage (V)")
plt.legend()
plt.grid()
plt.savefig(filename + "_volt_plot.png")
plt.show()

# Plot position
plt.figure()
voltage = df["Voltage"]
positions = []
for volt in voltage :
    position = ((volt * (old_voltage / new_voltage)) / old_voltage) * pot_length

```

```

        positions.append(position)
plt.plot(time, positions, label="acceleration")
plt.gca().xaxis.set_major_formatter(FuncFormatter(format_time))
plt.title("Position vs Time")
plt.xlabel("Time (hh:mm:ss)")
plt.ylabel("Position (cm)")
plt.legend()
plt.grid()
plt.savefig(filename + "_pos_plot.png")
plt.show()
plt.close()

# Plot Acceleration
plt.figure()

time = df["Time"]

acceleration = []
positions = []
for volt in voltage:
    position = ((volt * (old_voltage / new_voltage)) / old_voltage) * pot_length
    accl = ((-2 * spring_const * (position - pot_length/2)) / mass)
    acceleration.append(accl)

plt.plot(time, acceleration, label="acceleration")
plt.gca().xaxis.set_major_formatter(FuncFormatter(format_time))
plt.title("Acceleration vs Time")
plt.xlabel("Time (hh:mm:ss)")
plt.ylabel("Acceleration m/s^2")
plt.legend()
plt.grid()
plt.savefig(filename + "_acc_plot.png")
plt.show()
plt.close()

```